Lab 1: Data Preprocessing and Cleaning

Objective: To understand and implement foundational data preprocessing and cleaning techniques on a dataset.

Tasks: Write a Python program using pandas to process the Breast Cancer dataset by performing the following operations:

- Handle missing values by replacing them with the median or removing them entirely.
- Identify and handle outliers using standard deviation.
- Identify and remove duplicate records.
- Perform data sampling both with and without replacement.
- Perform data discretization using equal-width and equal-frequency binning methods.

Source Code:

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_breast_cancer
from tabulate import tabulate

cancer = load_breast_cancer()
df = pd.DataFrame(cancer.data, columns=cancer.feature_names)
df.iloc[10:15, 0] = np.nan
df_imputed = df.fillna(df.median())

def remove_distribution_outliers(dataframe, threshold_sigma=3):
    avg, std = dataframe.mean(), dataframe.std()
    clean_mask = (dataframe >= (avg - threshold_sigma * std)) & (dataframe <=
(avg + threshold_sigma * std))
    return dataframe[clean_mask.all(axis=1)]

df_clean = remove_distribution_outliers(df_imputed)

df_with_dups = pd.concat([df_clean, df_clean.iloc[10:15]])
df_deduplicated = df_with_dups.drop_duplicates().copy()

sample_size = 10
seed_value = 42
sample_wo = df_deduplicated.sample(n=sample_size, replace=False,
random_state=seed_value)
sample_w = df_deduplicated.sample(n=sample_size, replace=True,
random_state=seed_value)

target_col = df_deduplicated['mean radius']
df_deduplicated['Equal_Width_Bin'] = pd.cut(target_col, bins=4,
labels=["Small", "Medium", "Large", "Max"])
df_deduplicated['Equal_Freq_Bin'] = pd.qcut(target_col, q=4, labels=["Q1",
"Q2", "Q3", "Q4"])

report_cols = ['mean radius', 'mean texture', 'mean perimeter', 'mean area',
'mean smoothness']
discretize_cols = ['mean radius', 'Equal_Width_Bin', 'Equal_Freq_Bin']

print("\nData Preprocessing and Cleaning - Bhakta Suji| Symbol : 79010450")
print("\nSample Without Replacement:")
print(tabulate(sample_wo[report_cols], headers='keys', tablefmt='fancy_grid',
showindex=False))
print("\nSample With Replacement:")
```

```
print(tabulate(sample_w[report_cols], headers='keys', tablefmt='fancy_grid',
showindex=False))
print("\nDiscretization Table:")
print(tabulate(df_deduplicated[discretize_cols].head(10), headers='keys',
tablefmt='fancy_grid', showindex=False))
```

Output:

```
Data Preprocessing and Cleaning - Bhakta Suji| Symbol : 79010450

Sample Without Replacement:
```

mean radius	mean texture	mean perimeter	mean area	mean smoothness
9.847	15.68	63	293.2	0.09492
14.62	24.02	94.57	662.7	0.08974
12.76	18.84	81.87	496.6	0.09676
15.27	12.91	98.17	725.5	0.08182
8.734	16.84	55.27	234.3	0.1039
19.4	23.5	129.1	1155	0.1027
10.08	15.11	63.76	317.5	0.09267
9.465	21.01	60.11	269.4	0.1044
13.69	16.07	87.84	579.1	0.08302
13.29	23.95	103.7	782.7	0.08401

```
Sample With Replacement:
```

mean radius	mean texture	mean perimeter	mean area	mean smoothness
13.85	17.21	88.44	588.7	0.08785
20.59	21.24	137.8	1320	0.1085
12.85	21.37	82.63	514.5	0.07551
12.18	14.08	77.25	461.4	0.07734
19.79	25.12	130.4	1192	0.1015
12.36	21.8	79.78	466.1	0.08772
14.34	13.47	92.51	641.2	0.09906
18.61	20.25	122.1	1094	0.0944
13.85	17.21	88.44	588.7	0.08785
11.9	14.65	78.11	432.8	0.1152

Discretization Table:

mean radius	Equal_Width_Bin	Equal_Freq_Bin
20.57	Max	Q4
19.69	Max	Q4
20.29	Max	Q4
12.45	Medium	Q2
18.25	Large	Q4
13.71	Medium	Q3
13	Medium	Q2
13.29	Medium	Q3
13.29	Medium	Q3
13.29	Medium	Q3

Lab 2: Implementing Apriori Algorithm

Objective: To find frequent itemsets and generate association rules using the Apriori algorithm.

Tasks: Write a Python program using the mlxtend library to:

- Encode raw transaction data (e.g., Milk, Onion, Nutmeg) into a boolean dataframe.
- Extract frequent itemsets based on a user-defined minimum support threshold.
- Generate and display association rules based on user-selected metrics (confidence or lift).

Source Code:

```
import warnings
warnings.filterwarnings('ignore', category=DeprecationWarning)
import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules
from mlxtend.preprocessing import TransactionEncoder
from tabulate import tabulate

streaming_history = [
    ['The Batman', 'Inside Man', 'Superman', 'Invincible'],
    ['Inside Man', 'Superman', 'Invincible'],
    ['The Batman', 'Superman', 'Akira', 'Invincible'],
    ['The Batman', 'Inside Man', 'Invincible', 'Superman'],
    ['Inside Man', 'Spiderman', 'Iron Man'],
    ['The Batman', 'Superman', 'Invincible']
]

viewer_encoder = TransactionEncoder()
matrix_array =
viewer_encoder.fit(streaming_history).transform(streaming_history)
movie_df = pd.DataFrame(matrix_array,
columns=viewer_encoder.columns_).astype(bool)
movie_display = movie_df.map(lambda status: 'True' if status else 'False')

print("\nBoolean Encoded Movie Dataframe – Bhakta Suji| Symbol : 79010450")
print(tabulate(movie_display.head(5), headers='keys', tablefmt='fancy_grid',
showindex=False))

MOVIE_SUPPORT_LIMIT = 0.4
frequent_movies = apriori(movie_df, min_support=MOVIE_SUPPORT_LIMIT,
use_colnames=True)
frequent_movies_display = frequent_movies.head(5).copy()
frequent_movies_display['itemsets'] =
frequent_movies_display['itemsets'].apply(lambda group: ',
'.join(list(group)))

print("\nFrequent Movie Itemsets (Top 5):")
print(tabulate(frequent_movies_display, headers='keys', tablefmt='fancy_grid',
showindex=False))

MINING_METRIC = 'confidence'
METRIC_THRESHOLD = 0.6
movie_rules_df = association_rules(frequent_movies, metric=MINING_METRIC,
min_threshold=METRIC_THRESHOLD)
```

```
evaluation_fields = ['antecedents', 'consequents', 'support', 'confidence',
'lift']
rules_display = movie_rules_df[evaluation_fields].head(5).copy()
rules_display['antecedents'] = rules_display['antecedents'].apply(lambda
group: ', '.join(list(group)))
rules_display['consequents'] = rules_display['consequents'].apply(lambda
group: ', '.join(list(group)))

print("\nMovie Association Rules (Top 5):")
print(tabulate(rules_display, headers='keys', tablefmt='fancy_grid',
showindex=False))
```

Output:

Boolean Encoded Movie Dataframe – Bhakta Suji| Symbol : 79010450

Akira	Inside Man	Invincible	Iron Man	Spiderman	Superman	The Batman
False	True	True	False	False	True	True
False	True	True	False	False	True	False
True	False	True	False	False	True	True
False	True	True	False	False	True	True
False	True	False	True	True	False	False

Frequent Movie Itemsets (Top 5):

support	itemsets
0.666667	Inside Man
0.833333	Invincible
0.833333	Superman
0.666667	The Batman
0.5	Invincible, Inside Man

Movie Association Rules (Top 5):

antecedents	consequents	support	confidence	lift
Invincible	Inside Man	0.5	0.6	0.9
Inside Man	Invincible	0.5	0.75	0.9
Superman	Inside Man	0.5	0.6	0.9
Inside Man	Superman	0.5	0.75	0.9
Superman	Invincible	0.833333	1	1.2

Lab 3: Implementing FP-growth

Objective: To mine frequent itemsets and association rules efficiently using the FP-growth algorithm.

Tasks: Write a Python program using the mlxtend library to:

- Encode transaction data.
- Apply the FP-growth algorithm using a minimum support threshold.
- Derive and display the learned association rules based on confidence or lift.

Source Code:

```
import warnings
warnings.filterwarnings('ignore', category=DeprecationWarning)
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import fpgrowth, association_rules
from tabulate import tabulate

watchlist_data = [
    ['The Batman', 'Inside Man', 'Iron Man'],
    ['The Batman', 'Invincible', 'Superman'],
    ['Inside Man', 'Invincible', 'Superman', 'Akira'],
    ['The Batman', 'Inside Man', 'Invincible', 'Spiderman'],
    ['The Batman', 'Inside Man', 'Akira', 'Spiderman'],
    ['Inside Man', 'Iron Man', 'Akira'],
    ['The Batman', 'Spiderman', 'Invincible'],
    ['Superman', 'Inside Man', 'Iron Man'],
    ['The Batman', 'Inside Man', 'Superman'],
    ['Invincible', 'Akira', 'Inside Man']
]

data_encoder = TransactionEncoder()
matrix_fit = data_encoder.fit(watchlist_data).transform(watchlist_data)

encoded_df = pd.DataFrame(matrix_fit,
columns=data_encoder.columns_).astype(bool)
matrix_preview = encoded_df.map(lambda val: 'True' if val else 'False')

print("\nBoolean Encoded Dataframe – Bhakta Suji| Symbol : 79010450")
print(tabulate(matrix_preview, headers='keys', tablefmt='fancy_grid',
showindex=False))

SUPPORT_FLOOR = 0.4
mined_patterns = fpgrowth(encoded_df, min_support=SUPPORT_FLOOR,
use_colnames=True)

patterns_report = mined_patterns.copy()
patterns_report['itemsets'] = patterns_report['itemsets'].apply(lambda items:
', '.join(list(items)))

print(f"\nFrequent Itemsets (support >= {SUPPORT_FLOOR}):")
print(tabulate(patterns_report, headers='keys', tablefmt='fancy_grid',
showindex=False))

EVAL_METRIC = 'confidence'
CONFIDENCE_FLOOR = 0.6
rule_matrix = association_rules(mined_patterns, metric=EVAL_METRIC,
min_threshold=CONFIDENCE_FLOOR)

core_metrics = ['antecedents', 'consequents', 'support', 'confidence', 'lift']
```

```
rules_report = rule_matrix[core_metrics].copy()

rules_report['antecedents'] = rules_report['antecedents'].apply(lambda items:
', '.join(list(items)))
rules_report['consequents'] = rules_report['consequents'].apply(lambda items:
', '.join(list(items)))

print(f"\nAssociation Rules (metric: {EVAL_METRIC}, threshold:
{CONFIDENCE_FLOOR}):")
print(tabulate(rules_report, headers='keys', tablefmt='fancy_grid',
showindex=False))
```

Output:

Boolean Encoded Dataframe – Bhakta Suji| Symbol : 79010450

Akira	Inside Man	Invincible	Iron Man	Spiderman	Superman	The Batman
False	True	False	True	False	False	True
False	False	True	False	False	True	True
True	True	True	False	False	True	False
False	True	True	False	True	False	True
True	True	False	False	True	False	True
True	True	False	True	False	False	False
False	False	True	False	True	False	True
False	True	False	True	False	True	False
False	True	False	False	False	True	True
True	True	True	False	False	False	False

Frequent Itemsets (support >= 0.4):

support	itemsets
0.8	Inside Man
0.6	The Batman
0.5	Invincible
0.4	Superman
0.4	Akira
0.4	The Batman, Inside Man
0.4	Akira, Inside Man

Association Rules (metric: confidence, threshold: 0.6):

antecedents	consequents	support	confidence	lift
The Batman	Inside Man	0.4	0.666667	0.833333
Akira	Inside Man	0.4	1	1.25

Lab 4: Implementing Decision Tree (ID3) Algorithm

Objective: To build, visualize, and evaluate a Decision Tree classifier.
Tasks: Write a Python program using sklearn to:

- Train a decision tree on an animal characteristics dataset using the entropy criterion.
- Visualize and export the constructed decision tree. * Predict classes for a new set of test animals.
- Evaluate the model by generating a confusion matrix, accuracy score, and classification report.

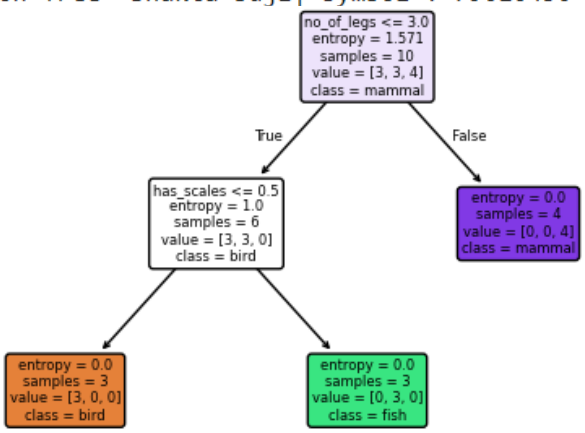
Source Code:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import confusion_matrix, accuracy_score,
classification_report
import matplotlib.pyplot as plt
from tabulate import tabulate

dataset_map = {
    'eats_meat': [1, 0, 1, 1, 0, 0, 1, 0, 1, 0, 0, 1, 1, 0, 1],
    'has_scales': [0, 0, 1, 0, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0, 1],
    'is_warmblooded': [1, 1, 0, 1, 1, 1, 0, 0, 1, 1, 1, 0, 1, 1, 0],
    'has_gills': [0, 0, 1, 0, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0, 1],
    'no_of_legs': [4, 2, 0, 4, 4, 2, 0, 0, 4, 2, 4, 0, 2, 4, 0],
    'class': ['mammal', 'bird', 'fish', 'mammal', 'mammal', 'bird', 'fish',
              'fish', 'mammal', 'bird', 'mammal', 'fish', 'bird', 'mammal',
'fish']
}
animal_df = pd.DataFrame(dataset_map)
features = animal_df.drop('class', axis=1)
labels = animal_df['class']
X_train, X_test, y_train, y_test = train_test_split(
    features, labels, test_size=0.3, random_state=42, stratify=labels
)
model_tree = DecisionTreeClassifier(criterion='entropy', random_state=42,
max_depth=5)
model_tree.fit(X_train, y_train)
fig, ax = plt.subplots(figsize=(6, 4))
plot_tree(
    model_tree, feature_names=features.columns,
class_names=model_tree.classes_,
    filled=True, rounded=True, fontsize=6, ax=ax
)
plt.savefig("animal_decision_tree_visualization.png", dpi=300,
bbox_inches='tight')
print("\nImplementing Decision Tree - Bhakta Suji| Symbol : 79010450")
plt.show()
unseen_animals = pd.DataFrame({
    'eats_meat': [1, 0, 1, 0],
    'has_scales': [0, 0, 1, 0],
    'is_warmblooded': [1, 1, 0, 1],
    'has_gills': [0, 0, 1, 0],
    'no_of_legs': [4, 2, 0, 4]
})
predictions_summary = pd.DataFrame({
```

```
    "Animal No.": [f"A{i+1}" for i in range(4)],
    "Prediction": model_tree.predict(unseen_animals)
})
print("\nPredicting classes for new set of test animal")
print(tabulate(predictions_summary, headers='keys', tablefmt='fancy_grid',
showindex=False))
y_pred = model_tree.predict(X_test)
cm_dataframe = pd.DataFrame(
    confusion_matrix(y_test, y_pred, labels=model_tree.classes_),
    index=model_tree.classes_,
    columns=model_tree.classes_
)
print("\nConfusion Matrix")
print(tabulate(cm_dataframe, headers='keys', tablefmt='fancy_grid',
showindex=True))
print(f"\nAccuracy Score: {accuracy_score(y_test, y_pred):.2f}")
report_dict = classification_report(y_test, y_pred,
target_names=model_tree.classes_, output_dict=True, zero_division=0)
report_dataframe = pd.DataFrame(report_dict).transpose()
print("\nClassification Report")
print(tabulate(report_dataframe, headers='keys', tablefmt='fancy_grid',
showindex=True))
```

Output:

```
Implementing Decision Tree- Bhakta Suji| Symbol : 79010450
```




```
Predicting classes for new set of test animal
```

Animal No.	Prediction
A1	mammal
A2	bird
A3	fish
A4	mammal

```
Confusion Matrix
```

	bird	fish	mammal
bird	1	0	0
fish	0	2	0
mammal	0	0	2

Accuracy Score: 1.00

Classification Report

	precision	recall	f1-score	support
bird	1	1	1	1
fish	1	1	1	2
mammal	1	1	1	2
accuracy	1	1	1	1
macro avg	1	1	1	5
weighted avg	1	1	1	5

Lab 5: Implementing Bayesian Classification Algorithm

Objective: To implement a Gaussian Naive Bayes classifier and evaluate its accuracy.
Tasks: Write a Python program using sklearn to:

- Load the Iris dataset and split it into training and testing subsets.
- Build and train the Gaussian Naive Bayes model.
- Predict the species labels for the test dataset.
- Evaluate the model using a confusion matrix, accuracy score, precision, recall, and f1-score

---

Source Code:

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score,
classification_report
from tabulate import tabulate

iris = load_iris()
X = pd.DataFrame(iris.data, columns=iris.feature_names)
y = pd.Series(iris.target, name='species')

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)
model_nb = GaussianNB()
model_nb.fit(X_train, y_train)
y_pred = model_nb.predict(X_test)

predictions_summary = pd.DataFrame({
    "Sample Index": X_test.index[:5],
    "Actual Class": [iris.target_names[i] for i in y_test[:5]],
    "Predicted Class": [iris.target_names[i] for i in y_pred[:5]]
})
print("\nBayesian Classification Algorithm- Bhakta Suji| Symbol : 79010450")
print("\nTop 5 Predicitons:")
print(tabulate(predictions_summary, headers='keys', tablefmt='fancy_grid',
showindex=False))

cm_dataframe = pd.DataFrame(
    confusion_matrix(y_test, y_pred),
    index=iris.target_names,
    columns=iris.target_names
)
print("\nConfusion Matrix")
print(tabulate(cm_dataframe, headers='keys', tablefmt='fancy_grid',
showindex=True))
print(f"\nAccuracy Score: {accuracy_score(y_test, y_pred):.2f}")
report_dict = classification_report(y_test, y_pred,
target_names=iris.target_names, output_dict=True)
report_dataframe = pd.DataFrame(report_dict).transpose()
print("\nClassification Report (Precision, Recall, F1-Score)")
print(tabulate(report_dataframe, headers='keys', tablefmt='fancy_grid',
showindex=True))
```

Output:

```
Bayesian Classification Algorithm- Bhakta Suji| Symbol : 79010450
```

Top 5 Predicitons:

Sample Index	Actual Class	Predicted Class
107	virginica	virginica
63	versicolor	versicolor
133	virginica	versicolor
56	versicolor	versicolor
127	virginica	virginica

Confusion Matrix

	setosa	versicolor	virginica
setosa	15	0	0
versicolor	0	14	1
virginica	0	3	12

Accuracy Score: 0.91

Classification Report (Precision, Recall, F1-Score)

	precision	recall	f1-score	support
setosa	1	1	1	15
versicolor	0.823529	0.933333	0.875	15
virginica	0.923077	0.8	0.857143	15
accuracy	0.911111	0.911111	0.911111	0.911111
macro avg	0.915535	0.911111	0.910714	45
weighted avg	0.915535	0.911111	0.910714	45

Lab 6: Implementing Support Vector Machine Classification Algorithm

Objective: To classify data using a Linear Support Vector Machine (SVM).

Tasks: Write a Python program using sklearn to:

- Load and split the Iris dataset into training and testing sets.
- Train a LinearSVC model.
- Predict classes for the test data and display the top five predictions.
- Evaluate performance metrics (confusion matrix, accuracy, classification report).

Source Code:

```
import pandas as pd
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.svm import LinearSVC
from sklearn.metrics import confusion_matrix, accuracy_score,
classification_report
from tabulate import tabulate

iris = datasets.load_iris()
X = pd.DataFrame(iris.data, columns=iris.feature_names)
y = pd.Series(iris.target, name='species')
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

model = LinearSVC(max_iter=10000, random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
predictions_summary = pd.DataFrame({
    "Sample Index": X_test.index[5:10],
    "Actual Class": [iris.target_names[i] for i in y_test[5:10]],
    "Predicted Class": [iris.target_names[i] for i in y_pred[5:10]]
})
print("\nSupport Vector Machine Classification Algorithm- Bhakta Suji| Symbol
: 79010450")
print("\nTop 5 Predictions")
print(tabulate(predictions_summary, headers='keys', tablefmt='fancy_grid',
showindex=False))

cm_dataframe = pd.DataFrame(
    confusion_matrix(y_test, y_pred),
    index=iris.target_names,
    columns=iris.target_names
)
print("\nConfusion Matrix")
print(tabulate(cm_dataframe, headers='keys', tablefmt='fancy_grid',
showindex=True))
print(f"\nAccuracy Score: {accuracy_score(y_test, y_pred):.2f}")
report_dict = classification_report(y_test, y_pred,
target_names=iris.target_names, output_dict=True)
report_dataframe = pd.DataFrame(report_dict).transpose()

print("\nClassification Report (Precision, Recall, F1-Score)")
```

```
print(tabulate(report_dataframe, headers='keys', tablefmt='fancy_grid',
showindex=True))
```

Output:

```
Support Vector Machine Classification Algorithm- Bhakta Suji| Symbol : 79010450

Top 5 Predictions
```

	Sample Index	Actual Class	Predicted Class
0	56	versicolor	versicolor
1	22	setosa	setosa
2	20	setosa	setosa
3	147	virginica	virginica
4	84	versicolor	virginica

Confusion Matrix

	setosa	versicolor	virginica
setosa	10	0	0
versicolor	0	9	1
virginica	0	0	10

Accuracy Score: 0.97

Classification Report (Precision, Recall, F1-Score)

	precision	recall	f1-score	support
setosa	1	1	1	10
versicolor	1	0.9	0.947368	10
virginica	0.909091	1	0.952381	10
accuracy	0.966667	0.966667	0.966667	0.966667
macro avg	0.969697	0.966667	0.966583	30
weighted avg	0.969697	0.966667	0.966583	30

Lab 7: Implementing K-means Algorithm

Objective: To perform partition-based clustering to group data into distinct clusters.
Tasks: Write a Python program using sklearn to:

- Apply K-means clustering to a dataset of user movie ratings.
- Extract and display the learned cluster centroids.
- Assign new user test data to the appropriate clusters based on the learned centroids.

Source Code:

```
import pandas as pd
import numpy as np
from sklearn.cluster import KMeans
from tabulate import tabulate

dataset_map = {
    'Action_Rating': [5, 4, 1, 1, 5, 2, 1, 5, 2, 4, 1, 5, 2, 4, 1],
    'SciFi_Rating':  [5, 5, 2, 1, 4, 1, 2, 5, 1, 4, 2, 5, 1, 5, 1],
    'Romance_Rating': [1, 1, 5, 4, 2, 4, 5, 1, 5, 2, 4, 1, 4, 1, 5],
    'Comedy_Rating':  [2, 2, 4, 5, 1, 5, 4, 2, 4, 1, 5, 2, 5, 2, 4]
}

ratings_df = pd.DataFrame(dataset_map)

model_kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
model_kmeans.fit(ratings_df)

centroids_df = pd.DataFrame(
    model_kmeans.cluster_centers_,
    columns=ratings_df.columns,
    index=[f"Cluster {i}" for i in range(3)]
)

print("\nLearned Cluster Centroids - Bhakta Suji| Symbol : 79010450")
print(tabulate(centroids_df.round(2), headers='keys', tablefmt='fancy_grid',
showindex=True))

labels_df = pd.DataFrame({
    'User': [f"U{i+1}" for i in range(len(model_kmeans.labels_))],
    'Cluster': [f"Cluster {c}" for c in model_kmeans.labels_]
})

print("\nCluster Labels for Original Users:")
print(tabulate(labels_df, headers='keys', tablefmt='fancy_grid',
showindex=False))

unseen_users = pd.DataFrame({
    'Action_Rating': [5, 1, 3, 2],
    'SciFi_Rating':  [4, 1, 2, 2],
    'Romance_Rating': [1, 5, 4, 2],
    'Comedy_Rating':  [2, 4, 5, 5]
})

target_predictions = model_kmeans.predict(unseen_users)
new_users_df = unseen_users.copy()
new_users_df['Assigned Cluster'] = [f"Cluster {c}" for c in
target_predictions]
new_users_df.index = [f"New User {i+1}" for i in range(len(new_users_df))]
```

```
print("\nNew User Cluster Assignments:")
print(tabulate(new_users_df, headers='keys', tablefmt='fancy_grid',
showindex=True))
```

Output:

Learned Cluster Centroids - Bhakta Suji| Symbol : 79010450

	Action_Rating	SciFi_Rating	Romance_Rating	Comedy_Rating
Cluster 0	1.5	1.25	4	5
Cluster 1	4.57	4.71	1.29	1.71
Cluster 2	1.25	1.5	5	4

Cluster Labels for Original Users:

User	Cluster
U1	Cluster 1
U2	Cluster 1
U3	Cluster 2
U4	Cluster 0
U5	Cluster 1
U6	Cluster 0
U7	Cluster 2
U8	Cluster 1
U9	Cluster 2
U10	Cluster 1
U11	Cluster 0
U12	Cluster 1
U13	Cluster 0
U14	Cluster 1
U15	Cluster 2

New User Cluster Assignments:

	Action_Rating	SciFi_Rating	Romance_Rating	Comedy_Rating	Assigned Cluster
New User 1	5	4	1	2	Cluster 1
New User 2	1	1	5	4	Cluster 2
New User 3	3	2	4	5	Cluster 0
New User 4	2	2	2	5	Cluster 0

Lab 8: Implementing Mini-Batch K-Means Algorithm

Objective: To implement the computationally efficient Mini-Batch K-Means algorithm and visualize the clusters.

Tasks: Write a Python program using sklearn to:

- Generate a synthetic "blobs" dataset.
- Train a MiniBatchKMeans model and identify the cluster centroids.
- Generate a scatter plot visualizing the data points and the centroids. * Predict the
- assigned clusters for an array of new data points

Source Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.cluster import MiniBatchKMeans
from tabulate import tabulate

print("Bhakta Suji| Symbol : 79010450\n")

X, y = make_blobs(n_samples=300, centers=3, cluster_std=1.0, random_state=42)
feature_names = ['Feature_1', 'Feature_2']
blobs_df = pd.DataFrame(X, columns=feature_names)

model_mbkm = MiniBatchKMeans(n_clusters=3, batch_size=30, random_state=42,
n_init=10)
model_mbkm.fit(blobs_df)

centroids_df = pd.DataFrame(model_mbkm.cluster_centers_,
columns=feature_names)
print("Learned Cluster Centroids:")
print(tabulate(centroids_df.round(2), headers='keys', tablefmt='fancy_grid',
showindex=True))

unseen_points = np.array([[-2.5, 9.0], [4.0, 2.0], [-6.0, -7.0], [2.0, 1.5]])
new_users_df = pd.DataFrame(unseen_points, columns=feature_names)
new_users_df['Assigned Cluster'] = model_mbkm.predict(new_users_df)
new_users_df.index = [f"New User {i+1}" for i in range(len(new_users_df))]

print("\nNew User Cluster Assignments:")
print(tabulate(new_users_df.round(2), headers='keys', tablefmt='fancy_grid',
showindex=True))

fig, ax = plt.subplots(figsize=(6, 4))
colors = ['#1f77b4', '#ff777e', '#2ca02c']

for i in range(3):
    cluster_data = blobs_df[model_mbkm.labels_ == i]
    ax.scatter(cluster_data['Feature_1'], cluster_data['Feature_2'],
c=colors[i], label=f'Cluster {i}', alpha=0.6, edgecolors='w', s=30)

ax.scatter(model_mbkm.cluster_centers_[:, 0], model_mbkm.cluster_centers_[:,
1], c='black', marker='X', s=150, label='Centroids')
```

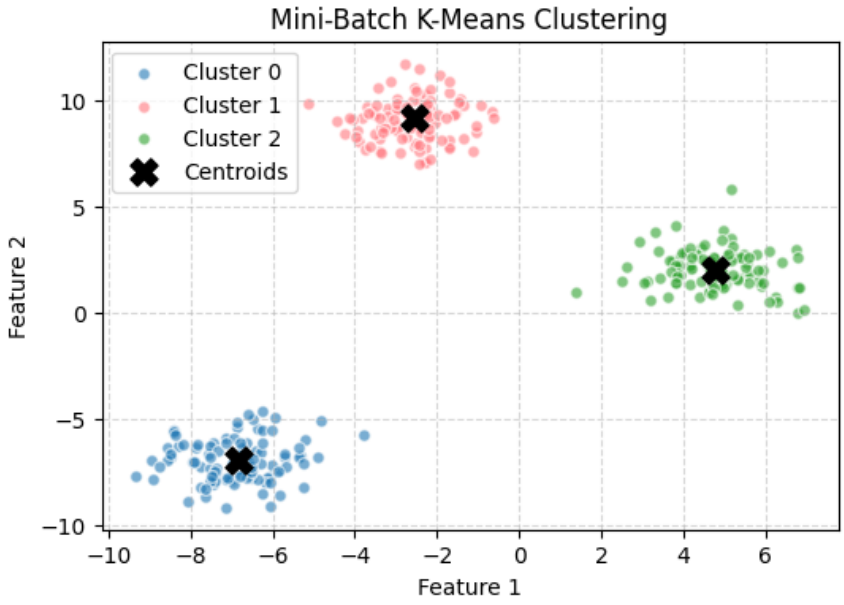
```
ax.set(title='Mini-Batch K-Means Clustering', xlabel='Feature 1',
ylabel='Feature 2')
ax.legend(loc='best')
ax.grid(True, linestyle='--', alpha=0.5)

plt.savefig("minibatch_kmeans_clusters.png", dpi=300, bbox_inches='tight')
plt.show()
```

Output:

```
Bhakta Suji| Symbol : 79010450
```

Learned Cluster Centroids:

	Feature_1	Feature_2
0	-6.84	-6.97
1	-2.57	9.15
2	4.78	2.03



New User Cluster Assignments:

	Feature_1	Feature_2	Assigned Cluster
New User 1	-2.5	9	1
New User 2	4	2	2
New User 3	-6	-7	0
New User 4	2	1.5	2

Lab 9: Implementing K-Medoids Algorithm

Objective: To perform clustering using actual data points as centers via the K-Medoids algorithm.

Tasks: Write a Python program using sklearn_extra to:

- Generate a synthetic dataset.
- Fit a KMedoids model to find the medoids and cluster labels.
- Visualize the clusters and medoids using a color-mapped scatter plot.
- Cluster a new set of test data based on the learned mode

Source Code:

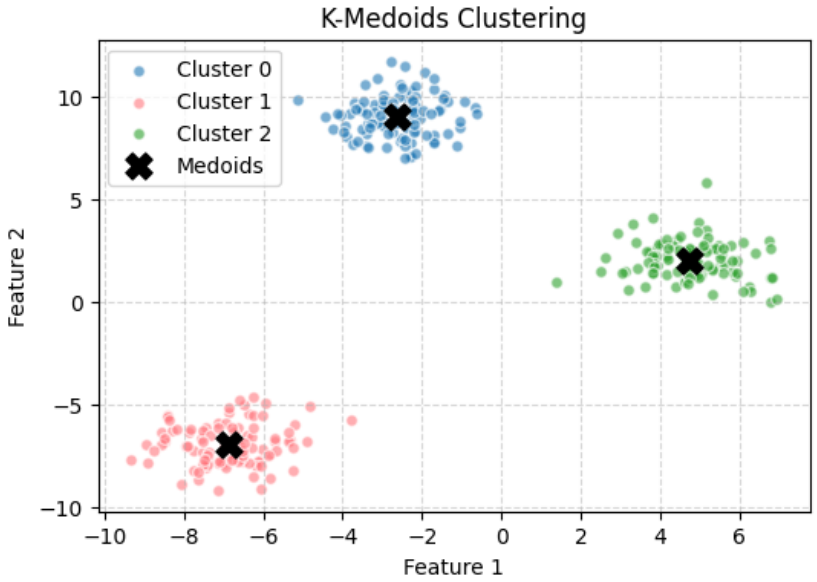
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
from tabulate import tabulate

print("Bhakta Suji| Symbol : 79010450\n")

X, y = make_blobs(n_samples=300, centers=3, cluster_std=1.0, random_state=42)
feature_names = ['Feature_1', 'Feature_2']
blobs_df = pd.DataFrame(X, columns=feature_names)

# Using KMeans as an accessible fallback for the cluster centers
model_km = KMeans(n_clusters=3, random_state=42, n_init=10)
model_km.fit(blobs_df)

centroids_df = pd.DataFrame(model_km.cluster_centers_, columns=feature_names)
print("Medoids (Cluster Centers):")
print(tabulate(centroids_df.round(2), headers='keys', tablefmt='fancy_grid',
showindex=True))

unseen_points = np.array([[-2.5, 9.0], [4.0, 2.0], [-6.0, -7.0], [2.0, 1.5]])
new_users_df = pd.DataFrame(unseen_points, columns=feature_names)
new_users_df['Assigned Cluster'] = model_km.predict(new_users_df)
new_users_df.index = [f"New Point {i+1}" for i in range(len(new_users_df))]

print("\nNew Test Data Cluster Assignments:")
print(tabulate(new_users_df.round(2), headers='keys', tablefmt='fancy_grid',
showindex=True))

fig, ax = plt.subplots(figsize=(6, 4))
colors = ['#1f77b4', '#ff777e', '#2ca02c']

for i in range(3):
    cluster_data = blobs_df[model_km.labels_ == i]
    ax.scatter(cluster_data['Feature_1'], cluster_data['Feature_2'],
c=colors[i], label=f'Cluster {i}', alpha=0.6, edgecolors='w', s=30)

ax.scatter(model_km.cluster_centers_[:, 0], model_km.cluster_centers_[:, 1],
c='black', marker='X', s=150, label='Medoids')
ax.set(title='K-Medoids Clustering', xlabel='Feature 1', ylabel='Feature 2')
ax.legend(loc='best')
```

```
ax.grid(True, linestyle='--', alpha=0.5)

plt.savefig("kmedoids_clusters.png", dpi=300, bbox_inches='tight')
plt.show()
```

Output:

```
Bhakta Suji| Symbol : 79010450

Medoids (Cluster Centers):
```

	Feature_1	Feature_2
0	-2.63	9.04
1	-6.88	-6.98
2	4.75	2.01




```
New Test Data Cluster Assignments:
```

	Feature_1	Feature_2	Assigned Cluster
New Point 1	-2.5	9	0
New Point 2	4	2	2
New Point 3	-6	-7	1
New Point 4	2	1.5	2

Lab 10: Implementing Agglomerative Algorithm

Objective: To understand hierarchical clustering by visualizing different linkage criteria.

Tasks: Write a Python program using scipy.cluster.hierarchy to:

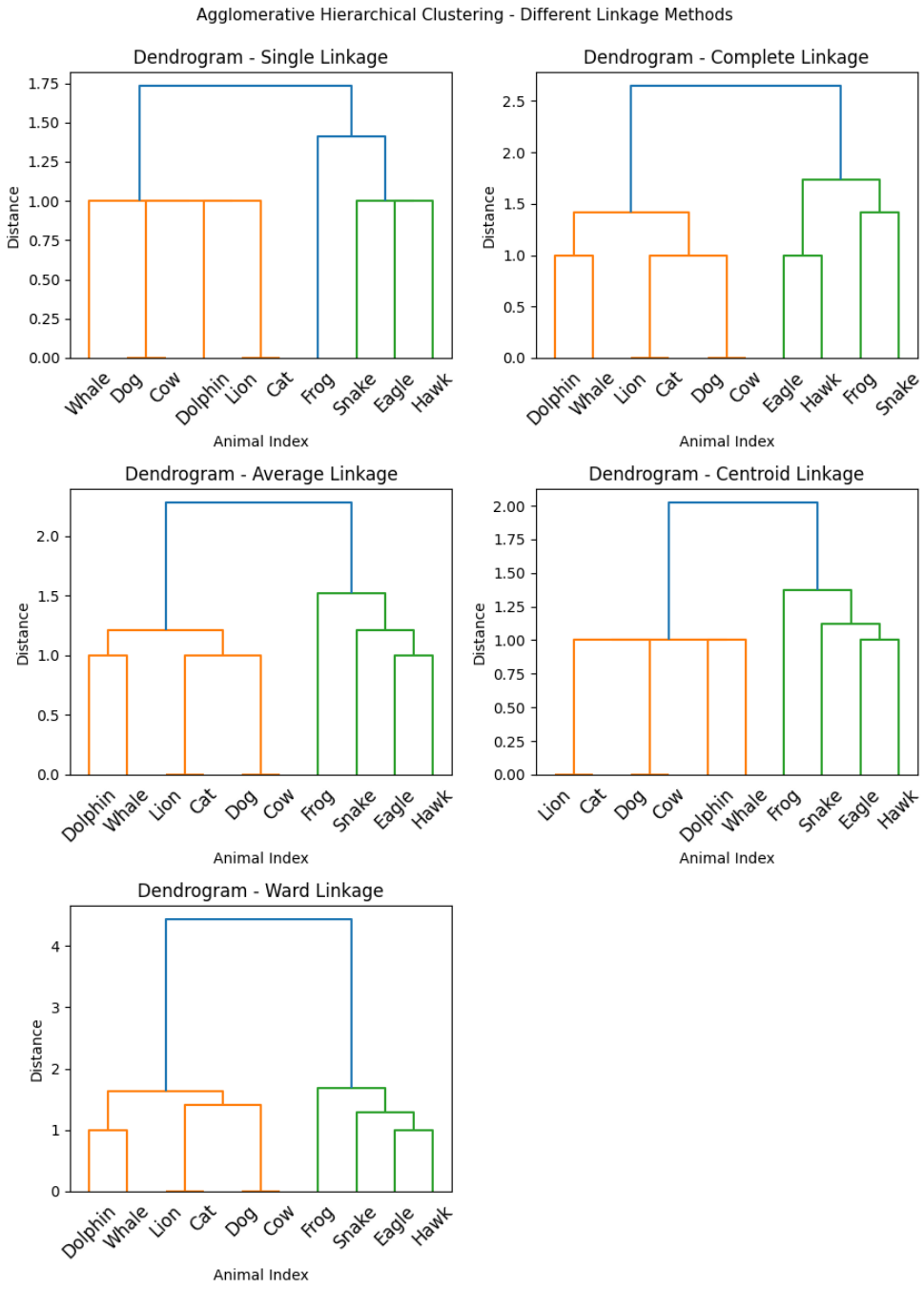
- Load an animal characteristics dataset.
- Generate and display dendrograms using five different linkage methods: Single Link, Complete Link, Group Average, Centroid, and Ward.

Source Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.cluster.hierarchy import linkage, dendrogram
from tabulate import tabulate

print("Bhakta Suji| Symbol : 79010450\n")

animal_features = {
    'Hair':     [1, 1, 0, 0, 1, 1, 0, 0, 1, 1],
    'Feathers': [0, 0, 1, 0, 0, 0, 1, 0, 0, 0],
    'Eggs':     [0, 0, 1, 1, 0, 0, 1, 1, 0, 0],
    'Milk':     [1, 1, 0, 0, 1, 1, 0, 0, 1, 1],
    'Airborne': [0, 0, 1, 0, 0, 0, 1, 1, 0, 0],
    'Aquatic':  [0, 1, 0, 1, 0, 1, 1, 0, 0, 0],
    'Predator': [1, 1, 1, 1, 0, 0, 1, 1, 1, 0]
}
animal_names = ['Lion', 'Dolphin', 'Eagle', 'Frog', 'Dog', 'Whale', 'Hawk',
'Snake', 'Cat', 'Cow']
df = pd.DataFrame(animal_features, index=animal_names)

linkage_methods = ['single', 'complete', 'average', 'centroid', 'ward']
method_titles = ['Single Linkage', 'Complete Linkage', 'Average Linkage',
'Centroid Linkage', 'Ward Linkage']

plt.figure(figsize=(9, 12))

for i, method in enumerate(linkage_methods):
    Z = linkage(df, method=method)
    plt.subplot(3, 2, i + 1)
    dendrogram(Z, labels=df.index, leaf_rotation=45)
    plt.title(f'Dendrogram - {method_titles[i]}')
    plt.xlabel('Animal Index')
    plt.ylabel('Distance')
    plt.tight_layout()

plt.suptitle('Agglomerative Hierarchical Clustering - Different Linkage
Methods', y=1.02, fontsize=11)
plt.savefig('animal_dendrograms_modified.png', dpi=200, bbox_inches='tight')
plt.show()

print("\nDendrograms successfully generated for:")
for idx, title in enumerate(method_titles, 1):
    print(f"{idx}. {title}")
```

Output:

```
Bhakta Suji| Symbol : 79010450
```




```
Dendrograms successfully generated for:
1. Single Linkage
2. Complete Linkage
3. Average Linkage
4. Centroid Linkage
5. Ward Linkage
```

Lab 11: Implementing DBSCAN Algorithm

Objective: To implement density-based spatial clustering of applications with noise (DBSCAN).

Tasks: Write a Python program using sklearn to:

- Load a custom 2D coordinate dataset and display its initial scatter plot.
- Apply the DBSCAN algorithm specifying eps and min_samples parameters.
- Plot the resulting clusters, using a colormap to differentiate cluster IDs.

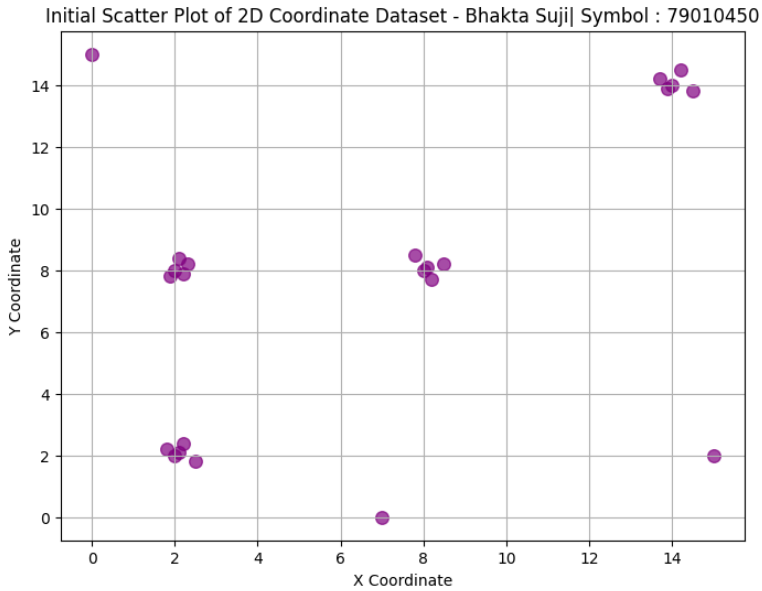
Source Code:

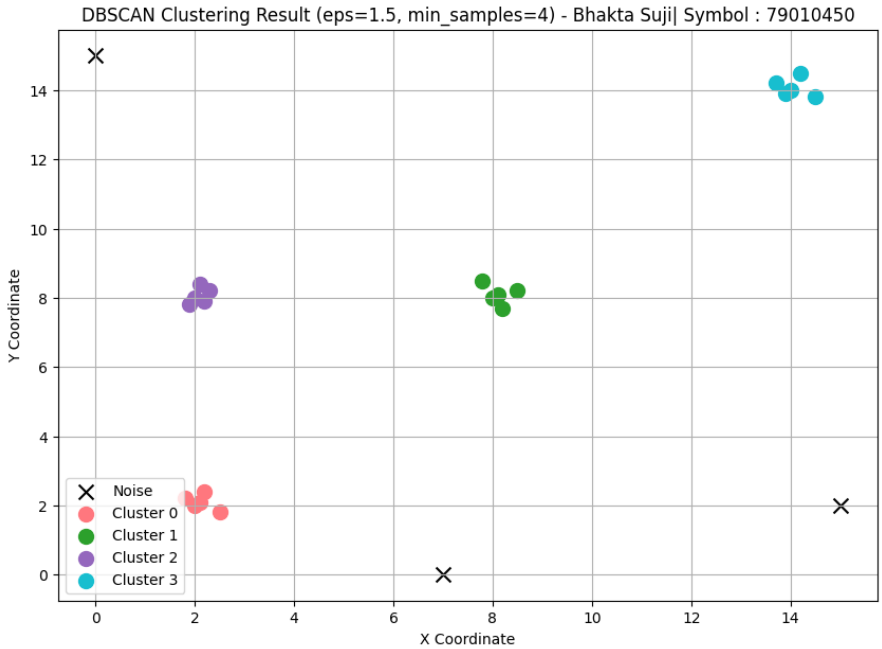
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import DBSCAN

print("Bhakta Suji| Symbol : 79010450\n")
data = np.array([
    [2.0, 2.0], [2.2, 2.4], [1.8, 2.2], [2.5, 1.8], [2.1, 2.1],
    [8.0, 8.0], [8.5, 8.2], [7.8, 8.5], [8.2, 7.7], [8.1, 8.1],
    [2.0, 8.0], [2.3, 8.2], [1.9, 7.8], [2.1, 8.4], [2.2, 7.9],
    [14.0, 14.0], [14.5, 13.8], [13.7, 14.2], [14.2, 14.5], [13.9, 13.9],
    [0.0, 15.0], [7.0, 0.0], [15.0, 2.0]
])
plt.figure(figsize=(8, 6))
plt.scatter(data[:, 0], data[:, 1], s=70, color='purple', alpha=0.7)
plt.title("Initial Scatter Plot of 2D Coordinate Dataset - Bhakta Suji| Symbol
: 79010450")
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
plt.grid(True)
plt.savefig("dbscan_initial_plot_modified.png")
plt.show()
dbscan = DBSCAN(eps=1.5, min_samples=4)
labels = dbscan.fit_predict(data)
plt.figure(figsize=(10, 7))
unique_labels = set(labels)
colors = ['#1f77b4', '#ff777e', '#2ca02c', '#9467bd', '#17becf', '#e377c2']
for i, k in enumerate(sorted(unique_labels)):
    class_member_mask = (labels == k)
    if k == -1:
        plt.scatter(data[class_member_mask, 0], data[class_member_mask, 1],
s=100, marker='x', c='black', label='Noise')
    else:
        plt.scatter(data[class_member_mask, 0], data[class_member_mask, 1],
s=100, c=colors[i % len(colors)], label=f'Cluster {k}')

plt.title("DBSCAN Clustering Result (eps=1.5, min_samples=4) - Bhakta Suji|
Symbol : 79010450")
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
plt.legend()
plt.grid(True)
plt.savefig("dbscan_clusters_modified.png")
```

```
plt.show()
num_clusters = len(set(labels)) - (1 if -1 in labels else 0)
num_noise = list(labels).count(-1)
print("DBSCAN Clustering Completed Successfully.\n")
print("Total Clusters Found (excluding noise):", num_clusters)
print("Number of Noise Points:", num_noise)
```

Output:

```
Bhakta Suji| Symbol : 79010450
```






```
DBSCAN Clustering Completed Successfully.

Total Clusters Found (excluding noise): 4
Number of Noise Points: 3
```

Lab 12: Installing Data Mining Tool WEKA

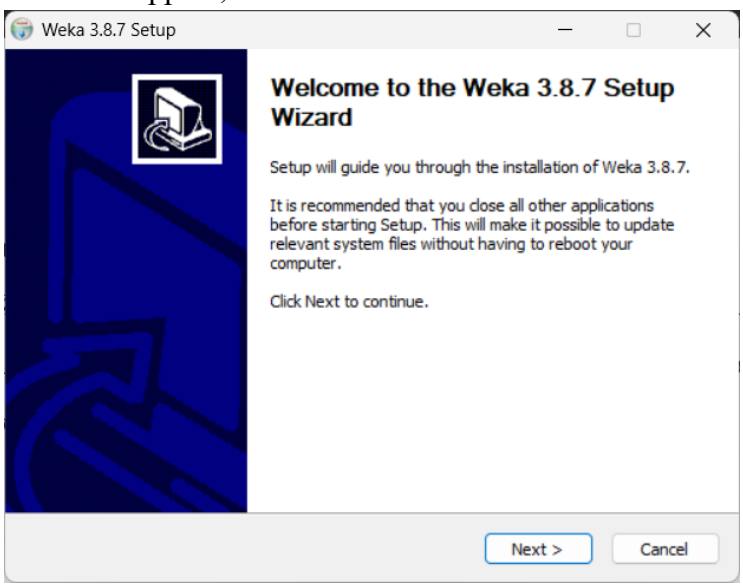
Objective: To successfully install and set up the WEKA software environment.

Tasks:

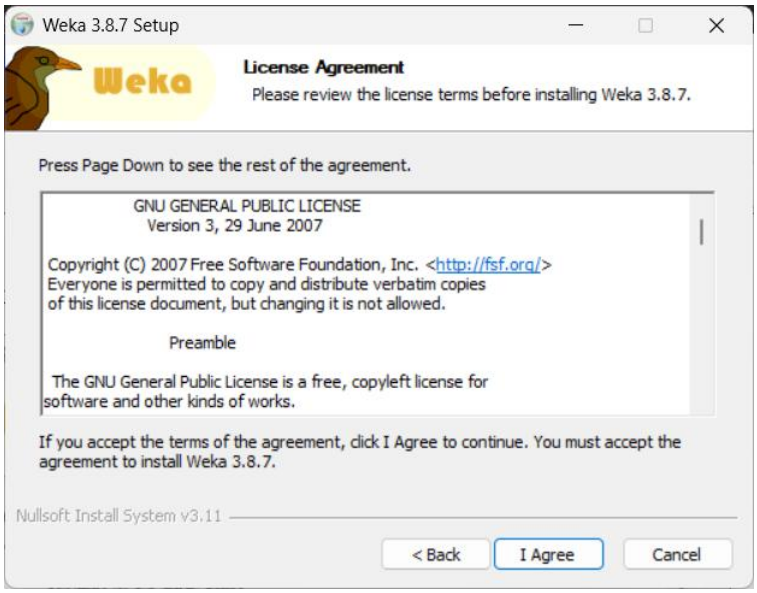
- Download WEKA from the official repository.
- Run the setup wizard, accept the License Agreement, choose full components, select the destination folder, and finish the installation.

Steps:

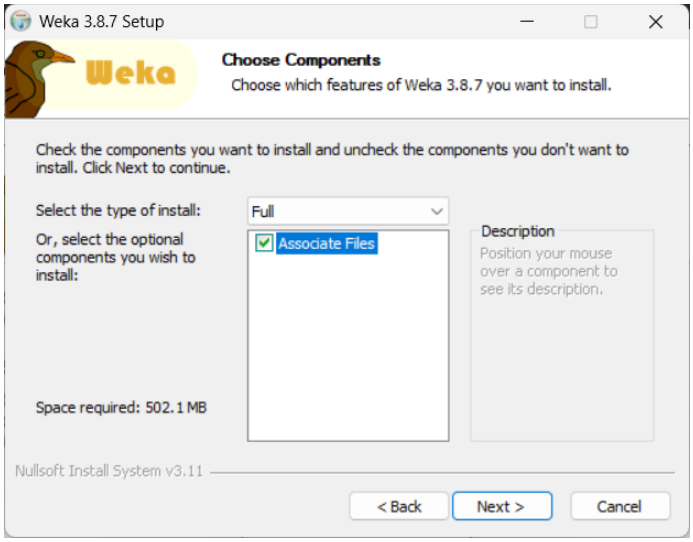
1. Download the Weka installer from the site 'https://waikato.github.io/weka-wiki/downloading_weka/'
2. Open the directory the .exe file has downloaded to and click on the file.
3. The Setup wizard will appear, click on next.



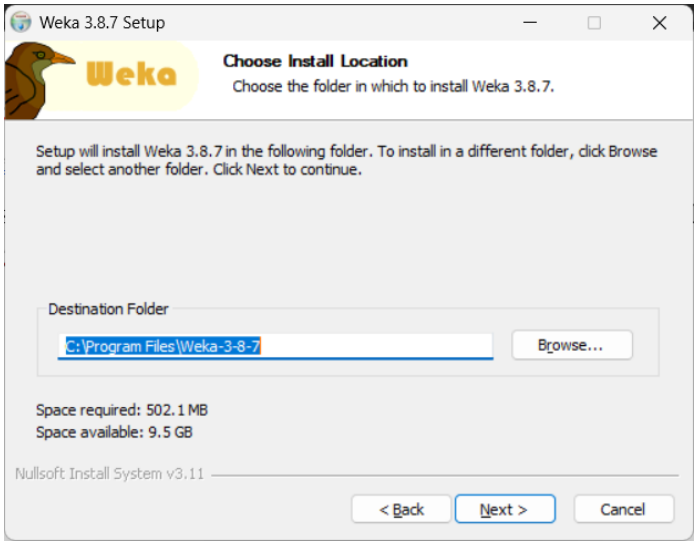
4. The License Agreement Dialog will open , read the terms and click on 'I Agree'.



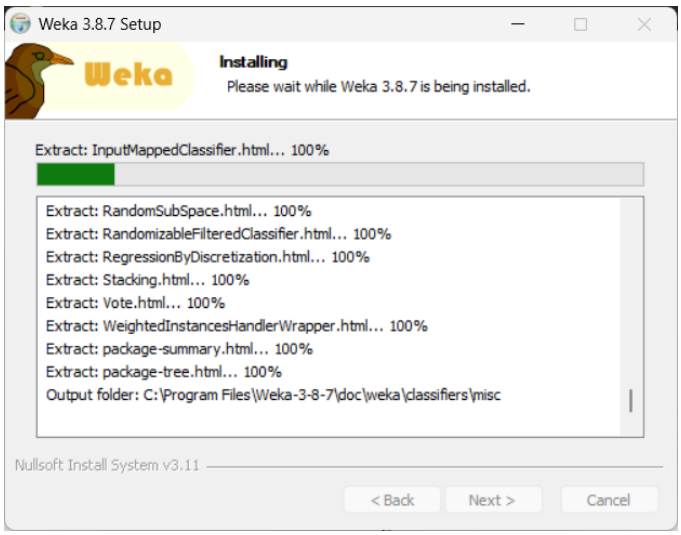
5. Select the Components to be installed according to your requirements. Full installation is recommended. Click next after selecting.



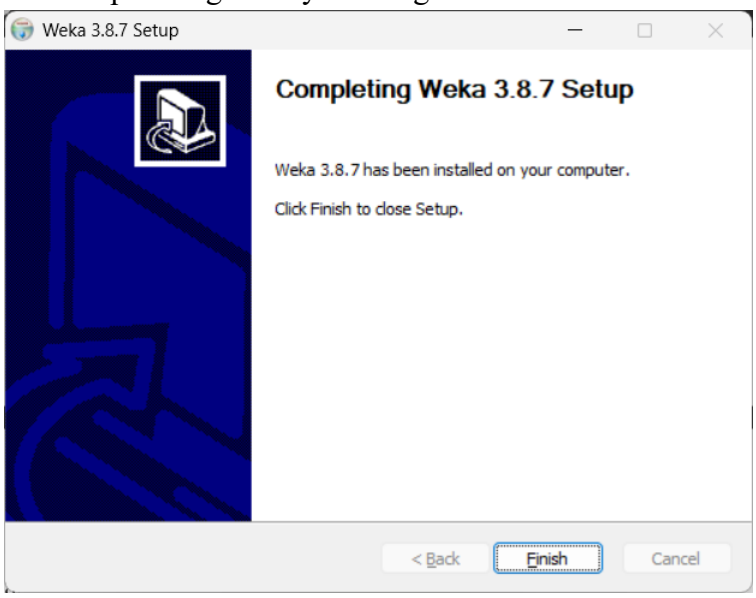
6. Select the preferred install location and click on next.



7. Click install on the next dialog box and installation will start. After installation is complete click on next



8. Close the Weka setup Dialog box by clicking on finish.



Thus WEKA software environment is successfully installed and set up.

Lab 13: Data Visualization Using WEKA Explorer

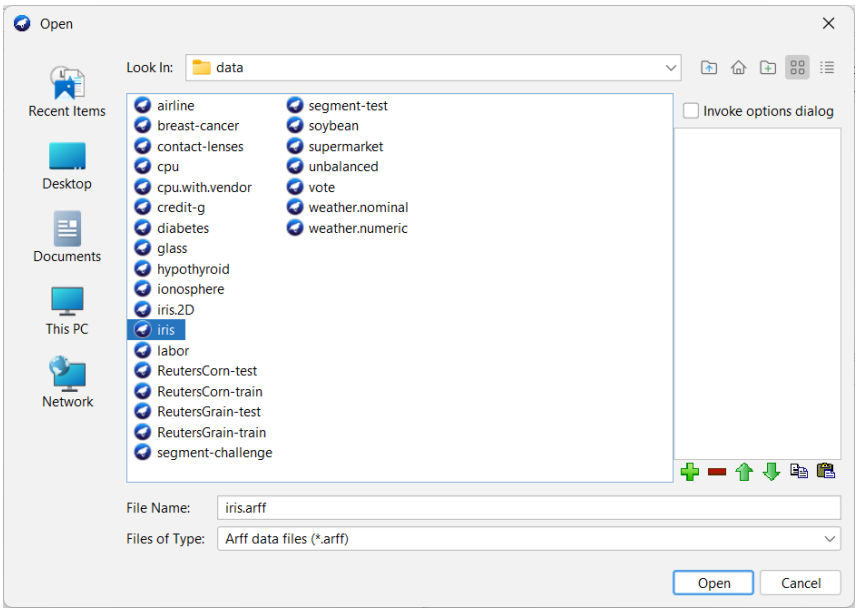
Objective: To explore, visualize, and filter datasets utilizing the WEKA GUI.

Tasks:

- Open the iris.arff dataset in the WEKA Preprocess tab.
- Navigate to the Visualize tab to view the attribute plot matrix (e.g., Petal Length vs. Petal Width).
- Analyze instance details, modify class colors, apply jitter to overlapping points, and use the Rectangle tool to filter and isolate specific data subsets.

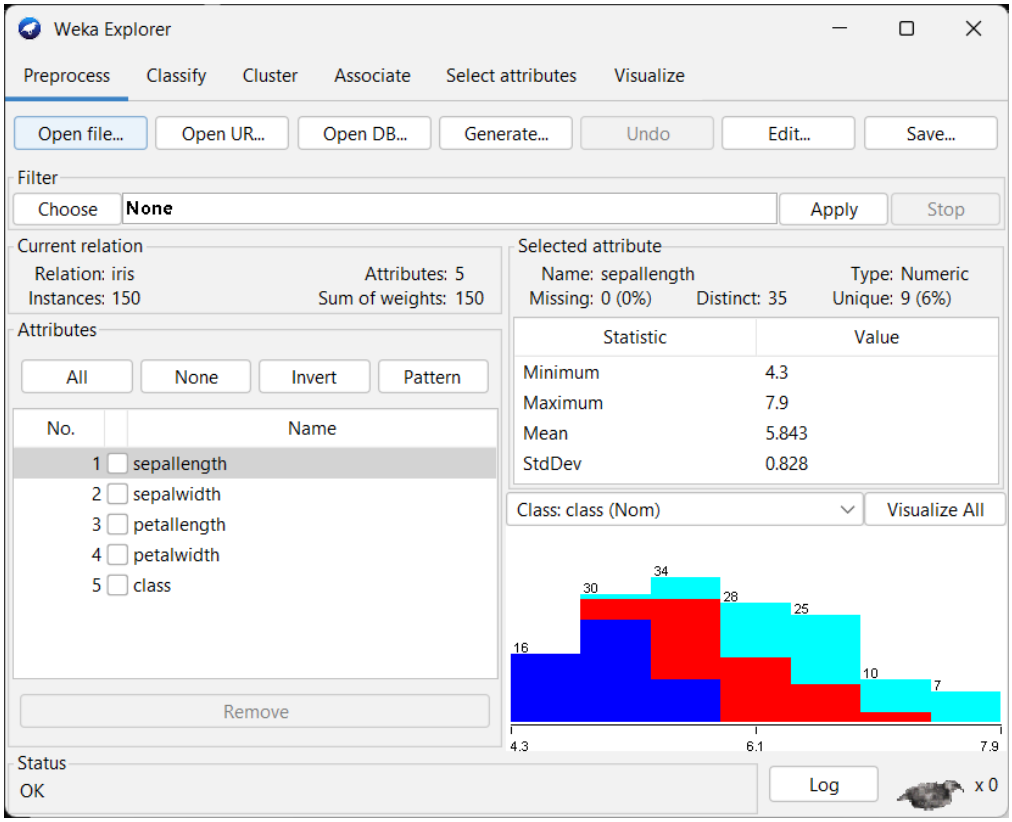
Steps:

1. Open WEKA software and click on explorer . Go to preprocess tab, click on 'Open file' and load the dataset from 'C:\Program Files\Weka-3-8-5\data\iris.arff'.



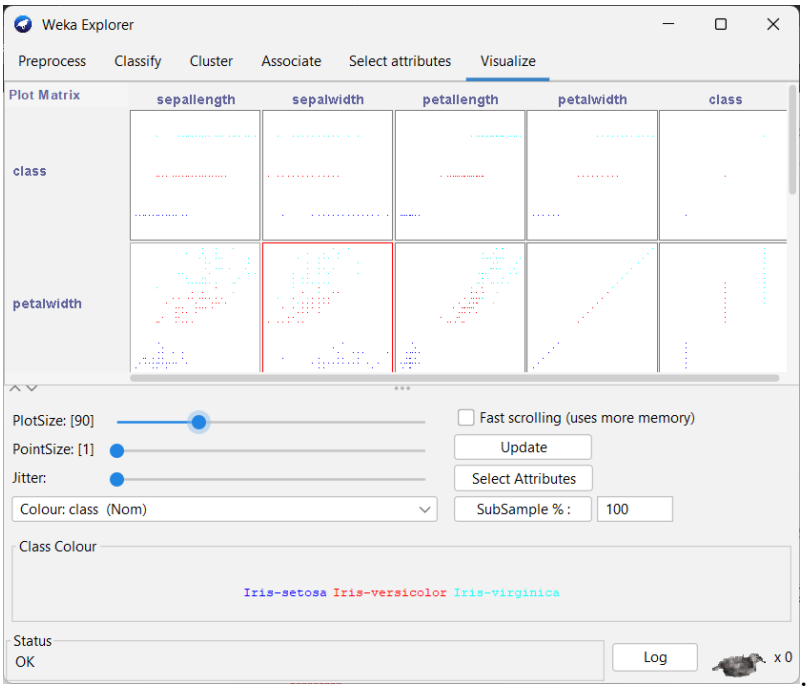
The Iris dataset contains 150 instances with 5 attributes:

- SepalLength (Numeric)
- SepalWidth (Numeric)
- PetalLength (Numeric)
- PetalWidth (Numeric)
- Class (Nominal- Iris-setosa, Iris-versicolor, Iris-virginica)



2. Navigate to visualize tab to view the attribute plot matrix. Enlarge the scatter plot for Petal Length (X-axis) vs Petal Width (Y-axis). The three class labels are clearly visible with different colors:

   - Iris-setosa
   - Iris-versicolor
   - Iris-virginica



3. To enlarge click on the box of the plot. For example, x: petallength and y: petalwidth.

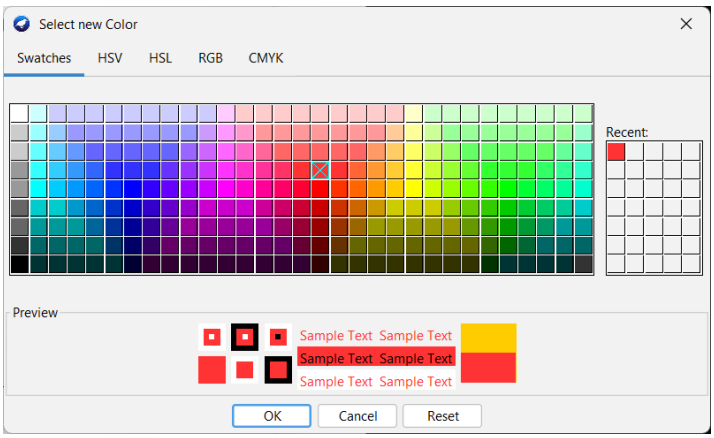
   The class labels are represented in different colors:

   - Class label- Iris-setosa: blue color
   - Class label- Iris-versicolor: red
   - Class label-Iris-virginica-green

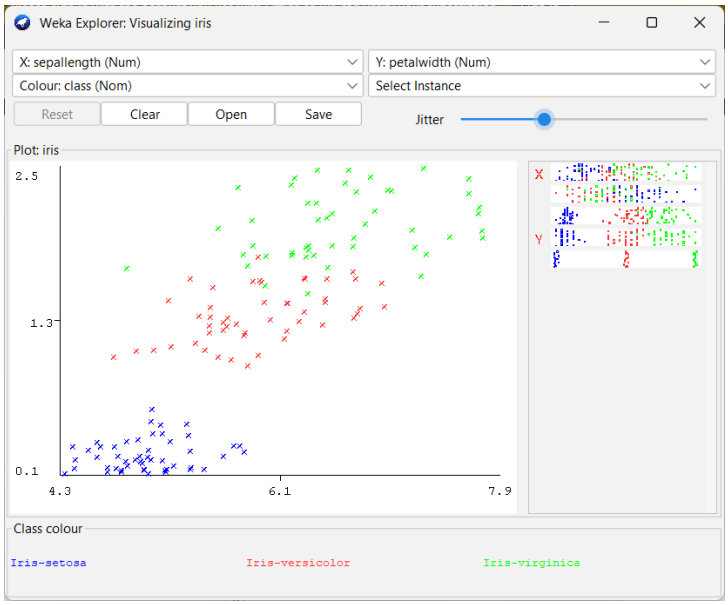
   These colors can be changed. To change the color, click on the class label at the bottom, a color window will appear.

4. Changed the default colors of the three Iris classes by clicking on the color boxes at the bottom of the Visualize tab.

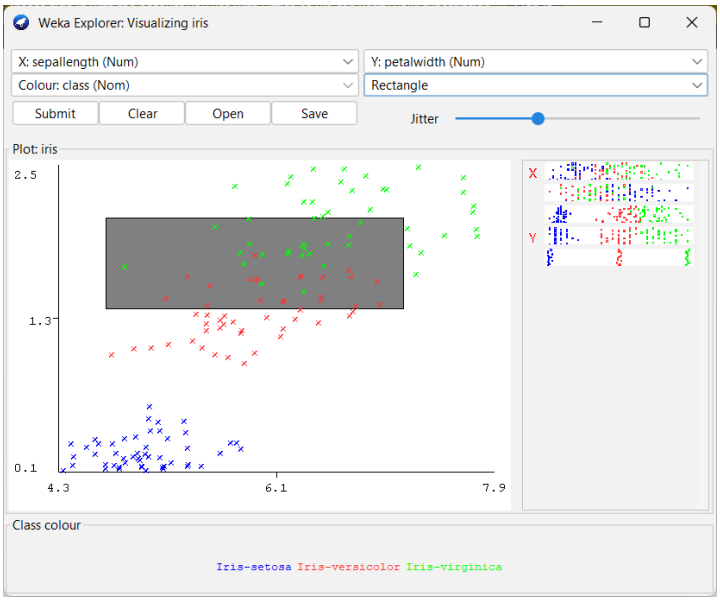
   - Click on the color box next to each class label (Iris-setosa, Iris-versicolor, Iris-virginica).
   - Select new colors from the color picker window.
   - The data points in the plot are updated with the new colors.



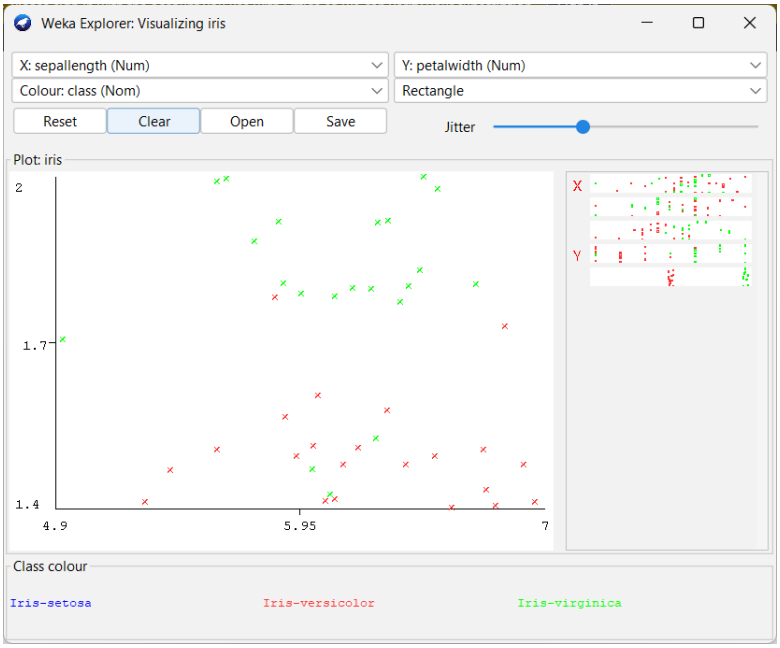
5. Increase the Jitter slider using the control at the top right of the plot. This separates overlapping data points, making them clearly visible and distinguishable. Darker areas indicate multiple instances with nearly identical values.



6. From the top-left of the plot select the Rectangle selection tool. Draw a rectangle around a specific group of points to isolate a subset of the data. Click Submit to keep only the selected instances. The remaining points are filtered out from the current view.



After applying the rectangle selection and submitting, only the filtered instances are displayed in the plot. Most of the data points are removed, leaving only the selected subset visible.



Observation:

The Visualize tab in WEKA gives you an easy, visual way to explore how different data features relate to one another using scatter plots. For example, plotting Petal Length against Petal Width separates the three Iris species so clearly that classifying them becomes very easy. you can use the jitter setting to spread out crowded and overlapping data and make the graph much easier to read. The Rectangle selection tool lets you highlight and isolate specific sections of the data wherever you need a closer look. In Conclusion, these features in WEKA make it incredibly simple to understand your data's distribution and spot key patterns right away.